手写小GPT学习笔记 | Day13

Day13承接Day12 FFN收尾工作，核心任务是**Transformer全组件串联搭建**，把此前8-12天手写的Scaled Dot-Product Attention、Mask、Multi-Head Attention、LayerNorm、残差连接、FFN六大核心零件，组装成可运行、可调试的完整Transformer Block，打通从零散组件到闭环模块的最后一环，为Day14周测跑通全流程做铺垫。

一、Day13 学习目标

- **组件复盘**：快速回顾各模块输入输出维度、接口规范，避免对接报错
- **串联逻辑**：理清Pre-LN架构、残差连接、掩码传递的核心流程
- **完整搭建**：整合所有手写组件，实现无Bug、可前向传播的Transformer模块
- **实战测试**：构造模拟输入+掩码，验证完整模块维度合规、运算正常
- **排坑总结**：解决组件串联常见的维度不匹配、掩码失效、梯度异常问题

二、前置准备：组件接口复盘（必看）

串联失败90%源于**维度不兼容、接口不统一**，先对齐所有组件的输入输出规则，统一超参数，杜绝前期报错。

1. 统一超参数（全局复用）

- **d_model**：模型特征维度，建议512/768，全程固定不变
- **num_heads**：注意力头数，必须整除d_model（如8头对应512维、12头对应768维）
- **dropout**：Dropout概率，统一设为0.1
- **expansion_factor**：FFN升维因子，固定为4

2. 各组件输入输出维度（核心对照）

组件名称	输入shape	输出shape	关键参数
Scaled Dot-Product Attention	q/k/v: [batch, heads, seq, d_k]	output: [batch, heads, seq, d_k]	mask: [batch, 1, seq_q, seq_k]
Multi-Head Attention			num_heads, mask

	q/k/v: [batch, seq, d_model]	output: [batch, seq, d_model]	
LayerNorm	[batch, seq, d_model]	[batch, seq, d_model]	d_model
FFN	[batch, seq, d_model]	[batch, seq, d_model]	expansion_factor

> 💡 **红线规则**：所有组件输入输出必须保持**[batch_size, seq_len, d_model]** 三维结构，残差连接才能正常相加！

三、Transformer完整架构（Pre-LN版）

采用GPT标配的**Pre-LN架构**（先归一化、再运算），相比Post-LN更稳定，不易梯度消失，串联流程如下：

1. **注意力子层**：LayerNorm → 多头注意力（带Mask）→ Dropout → 残差相加
2. **FFN子层**：LayerNorm → FFN → Dropout → 残差相加

全程**维度守恒**，每一步输出都和输入维度一致，保证模块可堆叠、可复用。

四、全组件串联手写代码（完整版）

整合Day8-Day12所有手写组件，无外部依赖、无封装黑箱，逐段注释对接前文，复制即可运行。

1. 基础依赖 + 工具函数（Mask）

Day13 基础依赖+掩码工具

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import os

# Mac MPS兼容配置
os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"

# -------------------------- 复用Day9：Mask掩码实现 --------------------------
def create_padding_mask(seq: torch.Tensor, pad_idx: int = 0) -> torch.Tensor:
    """填充掩码：屏蔽pad占位符，shape [batch, 1, 1, seq_len]"""
    mask = (seq == pad_idx).unsqueeze(1).unsqueeze(2)
    return mask

```

```python
def create_causal_mask(seq_len: int) -> torch.Tensor:
    """因果掩码（解码器专用）：屏蔽未来信息，下三角矩阵，shape [1,1,seq_len,seq_len]"""
    mask = torch.tril(torch.ones(seq_len, seq_len, dtype=torch.bool))
    return mask.unsqueeze(0).unsqueeze(0)

# -------------------------- 复用Day8：缩放点积注意力 --------------------------
class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout: float = 0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

    def forward(self, q: torch.Tensor, k: torch.Tensor, v: torch.Tensor, mask: torch.Tensor = None):
        d_k = q.size(-1)
        # 计算注意力分数
        attn_scores = torch.matmul(q, k.transpose(-2, -1)) / torch.sqrt(torch.tensor(d_k, dtype=torch.float32))
        # 掩码屏蔽（置为极小值）
        if mask is not None:
            attn_scores = attn_scores.masked_fill(mask == False, -1e9)
        # 注意力权重+dropout
        attn_weights = F.softmax(attn_scores, dim=-1)
        attn_weights = self.dropout(attn_weights)
        # 加权求和
        output = torch.matmul(attn_weights, v)
        return output, attn_weights

# -------------------------- 复用Day10：多头注意力 --------------------------
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model: int, num_heads: int, dropout: float = 0.1):
        super().__init__()
        assert d_model % num_heads == 0, "d_model必须能被num_heads整除"
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        # QKV投影层
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        # 输出投影
        self.w_o = nn.Linear(d_model, d_model)
        # 缩放点积注意力
        self.attn = ScaledDotProductAttention(dropout)
        self.dropout = nn.Dropout(dropout)

```

```python
    def split_heads(self, x: torch.Tensor, batch_size: int):
        """拆分多头：[batch, seq, d_model] → [batch, heads, seq, d_k]"""
        x = x.view(batch_size, -1, self.num_heads, self.d_k)
        return x.transpose(1, 2)

    def combine_heads(self, x: torch.Tensor, batch_size: int):
        """拼接多头：[batch, heads, seq, d_k] → [batch, seq, d_model]"""
        x = x.transpose(1, 2).contiguous()
        return x.view(batch_size, -1, self.d_model)

    def forward(self, q: torch.Tensor, k: torch.Tensor, v: torch.Tensor, mask: torch.Tensor = None):
        batch_size = q.size(0)
        # QKV线性投影
        q_proj = self.split_heads(self.w_q(q), batch_size)
        k_proj = self.split_heads(self.w_k(k), batch_size)
        v_proj = self.split_heads(self.w_v(v), batch_size)
        # 注意力计算
        attn_out, attn_weights = self.attn(q_proj, k_proj, v_proj, mask)
        # 拼接+输出投影
        out = self.combine_heads(attn_out, batch_size)
        out = self.dropout(self.w_o(out))
        return out, attn_weights

# ------------------------- 复用Day12：FFN前馈网络 -------------------------
class FeedForwardNetwork(nn.Module):
    def __init__(self, d_model: int, dropout: float = 0.1, expansion_factor:
```

2. 核心：完整Transformer Block（串联所有组件）

Day13 Transformer Block（全组件串联）

```python
class TransformerBlock(nn.Module):
    def __init__(self, d_model: int, num_heads: int, dropout: float = 0.1):
        super().__init__()
        # 1. 多头注意力（Day8+9+10）
        self.attn = MultiHeadAttention(d_model, num_heads, dropout)
        # 2. FFN（Day12）
        self.ffn = FeedForwardNetwork(d_model, dropout)
        # 3. LayerNorm（Day11）
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        # 4. Dropout+残差（Day11）
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor, mask: torch.Tensor = None):
        """
        前向传播（Pre-LN架构）
```

```
        Args:
            x: 输入特征 [batch_size, seq_len, d_model]
            mask: 组合掩码 [batch_size, 1, seq_len, seq_len]
        Returns:
            模块输出 [batch_size, seq_len, d_model]
        """
        # -------------------------- 注意力子层+残差 --------------------------
        residual = x
        # 先归一化
        x = self.norm1(x)
        # 多头注意力计算
        attn_out, _ = self.attn(x, x, x, mask)
        # Dropout+残差相加
        x = residual + self.dropout(attn_out)

        # -------------------------- FFN子层+残差 --------------------------
        residual = x
        # 先归一化
        x = self.norm2(x)
        # FFN特征加工
        ffn_out = self.ffn(x)
        # Dropout+残差相加
        x = residual + self.dropout(ffn_out)

        return x
```

3. 堆叠式Transformer（可选，进阶搭建）

堆叠多个Block（模拟GPT架构）

```
class TransformerEncoder(nn.Module):
    def __init__(self, d_model: int, num_heads: int, num_layers: int = 2, dropout: float = 0.1):
        super().__init__()
        # 堆叠多个Transformer Block
        self.layers = nn.ModuleList([
            TransformerBlock(d_model, num_heads, dropout) for _ in range(num_layers)
        ])
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x: torch.Tensor, mask: torch.Tensor = None):
        # 逐层传播
        for layer in self.layers:
            x = layer(x, mask)
        # 最终归一化
```

```
        return self.norm(x)
```

五、实战测试：验证串联有效性

构造模拟序列、生成掩码，测试完整模块是否正常运行、维度合规，排查串联bug。

Day13 全模块测试脚本

```
if __name__ == "__main__":
    # 设备适配
    device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu")
    print(f"测试设备：{device}")

    # 统一超参数
    batch_size = 2
    seq_len = 6
    d_model = 512
    num_heads = 8
    pad_idx = 0

    # 1. 构造模拟输入序列（含pad占位符）
    x = torch.randint(low=1, high=100, size=(batch_size, seq_len), device=device)
    x[0, 4:] = pad_idx  # 手动添加pad
    print(f"输入序列shape：{x.shape}")

    # 2. 生成掩码（padding+因果）
    padding_mask = create_padding_mask(x, pad_idx).to(device)
    causal_mask = create_causal_mask(seq_len).to(device)
    # 组合掩码
    mask = padding_mask & causal_mask
    print(f"组合掩码shape：{mask.shape}")

    # 3. 初始化完整模块
    transformer_block = TransformerBlock(d_model, num_heads).to(device)
    # 构造特征输入（模拟Embedding输出）
    feat = torch.randn(batch_size, seq_len, d_model, device=device)

    # 4. 前向传播
    with torch.no_grad():
        output = transformer_block(feat, mask)

    # 5. 核心验证：维度守恒
    print(f"模块输入特征shape：{feat.shape}")
```

```
    print(f"模块输出特征shape：{output.shape}")
    assert feat.shape == output.shape, "❌ 串联失败：维度不匹配"
    print("✅ Transformer全组件串联测试通过！")
```

六、Day13 串联避坑指南

💡 **高频报错+解决方案**

- **维度不匹配**：检查多头注意力拆分/拼接逻辑、FFN升维降维维度，严格对齐[batch, seq, d_model]格式
- **掩码失效**：掩码shape必须为[batch, 1, seq_len, seq_len]，注意布尔掩码和数值掩码的转换
- **残差报错**：残差分支和主分支维度必须完全一致，Pre-LN顺序不可颠倒
- **梯度NaN**：降低学习率、检查掩码极小值（-1e9而非-∞）、确认Xavier初始化生效
- **MPS报错**：掩码张量转为bool类型，避免浮点型掩码在MPS上不兼容

七、Day13 学习总结 + 后续衔接

✅ 今日完成：Transformer六大核心组件全串联 + 完整Block搭建 + 实战测试
✅ 核心掌握：Pre-LN架构逻辑、组件对接规范、掩码传递流程、残差连接机制

后续学习规划（Day14周测）

- **周测任务**：复现Day8-Day13所有内容，独立跑通完整Attention模块，默写核心公式和代码逻辑
- **复盘重点**：梳理组件串联流程、易错点、调试技巧，形成完整笔记闭环
- **进阶方向**：添加Embedding层、位置编码，搭建极简GPT雏形

🎯 至此Transformer核心模块已完全手写打通，Day14周测只需查漏补缺、巩固流程，即可完成第一阶段小目标！

（注：文档部分内容可能由 AI 生成）